Explaining the advantage of quantum-enhanced physics-informed neural networks

Nils Klement,[1] Veronika Eyring,[1,2] and Mierk Schwabe[1]

[1] *Deutsches Zentrum für Luft- und Raumfahrt, Institut für Physik der Atmosphäre, Oberpfaffenhofen, Germany*
[2] *University of Bremen, Institute of Environmental Physics (IUP), Bremen, Germany*

(Dated: January 22, 2026)

Partial differential equations (PDEs) form the backbone of simulations of many natural phenomena, for example in climate modeling, material science, and even financial markets. The application of physics-informed neural networks to accelerate the solution of PDEs is promising, but not competitive with numerical solvers yet. Here, we show how quantum computing can improve the ability of physics-informed neural networks to solve partial differential equations. For this, we develop hybrid networks consisting of quantum circuits combined with classical layers and systematically test them on various non linear PDEs and boundary conditions in comparison with purely classical networks. We demonstrate that the advantage of using quantum networks lies in their ability to achieve an accurate approximation of the solution in substantially fewer training epochs, particularly for more complex problems. These findings provide the basis for targeted developments of hybrid quantum neural networks with the goal to significantly accelerate numerical modeling.

arXiv:2601.15046v1 [quant-ph] 21 Jan 2026

A key challenge in simulating real-world phenomena is the high computational demand to solve the underlying partial differential equations (PDEs). This leads not only to the need for high performance computational centers and large power consumption [1, 2], but also limits the achievable resolutions. This is especially the case when real world data, such as observations or objectives, need to be included, and iterative numerical methods are therefore required. These requirements can be addressed in a unified manner by an approach called physics-informed neural network (PINN) [3]. For this purpose, not only the dynamics of the system, as described by a PDE, are incorporated into the loss function, but also arbitrary fixed points in space and time can be given, e.g. boundary conditions, observations, or objectives. An additional advantage of PINNs is that no discretization of space and time is needed, thus preventing discretization errors. Besides that, material properties [4] or parametrized geometries [5] can be treated as additional input variables, enabling the solution of a whole family of PDEs and the replacement of costly iterative methods. These features are especially relevant when the goal is to model real world phenomena supported by real world observations. For example, in climate models the fundamental dynamics are described by Navier-Stokes equations, while subgrid-scale processes are treated by parametrizations. These depend on free (tunable) parameters [6]. To identify the optimal set of parameters, climate models must be calibrated against real-world observations [7] which is commonly referred to as "the inverse problem" in the context of PINNs [8]. Given the current urgency of enhancing climate models at present [9], data-driven methods present promising advantages for this purpose. Another recent powerful application addresses the Navier–Stokes existence and smoothness [10, 11], one of the seven Clay Millennium Prize Problems.

Although in theory the possibilities and potential of classical PINNs (cPINNs) are promising, in reality they can not replace traditional numerical solvers yet. This is mostly due to trainability - or in other words the computational costs - and a related lack of performance [12, 13]. In this study, we provide insights into the performance of quantum neural networks (QNNs) in solving problems described by PDEs by integrating them into PINNs. We use the properties of QNNs as universal function approximators [14] to learn the solution of a given PDE by training a quantum physics-informed neural network (qPINN). Previous studies [15–19] indicate that in some cases using the potential of QNNs [20–23] in qPINNs can lead to an improved performance, finding better trainability [17] and reduction of final loss [16, 18, 19]. Advantages of this approach compared to other quantum approaches where the solutions are stored in a large state vectors [24, 25] is that no linearization of the PDE is necessary and that the readout problem can be avoided once the network is trained.

In this paper, we show that the advantage of qPINNs stems from their ability to find a good approximation of the solution much faster - in terms of training epochs - than cPINNs, simultaneously explaining the apparent reduced final losses of qPINNs. Although both network types allow for comparable approximations after sufficient training epochs, the faster convergence of qPINNs addresses the bottleneck of cPINNs. We show this by performing extensive training with up to $2 \cdot 10^4$ epochs for qPINNs and $1 \cdot 10^6$ epochs for cPINNs, taking the potentially highly complex loss landscape of PINNs into account [26]. Moreover, we evaluate the conditions under which PINN solutions benefit from including parametrized quantum circuits. Two different network types are considered. The first type is a dense purely classical network, the second type is a quantum-classical (hybrid) architecture. A comprehensive overview of the performance of the qPINNs is provided by solving parametrized PDEs with generic boundary conditions.

Information about the formulation of PINN losses, training settings, update of training data, and adaptive loss weights are given in the Supplemental Material [27], together with a detailed description of the architectures used, further results, and training insights as well as a brief overview on the technical implementation.

System definition— The PINN loss can be formulated based on two terms

$$\mathcal{L}_{\boldsymbol{\theta}} = w_{bounds}\mathcal{L}_{bounds} + w_{pde}\mathcal{L}_{pde}. \tag{1}$$

The fist term describes fixed points in space and time (bounds) and the second the dynamics of the system (pde). w_{bounds} and w_{pde} are weights of the individual loss terms determined as described later on. The domain, enclosed by the boundary conditions and governed by the PDE, has temporal coordinate t and spatial coordinate x. Since temporal u_t and spatial boundary conditions u_x are incorporated into the loss term $\mathcal{L}_{bounds}$ by the same mathematical description (Eq. (11) in the Supplemental Material [27]), the temporal boundary conditions are considered representative. Without loss of generality the spatial boundary conditions are chosen depending on the temporal boundary conditions by fixing $u_x(t, x = 0) = u_t(x = 0)$ and $u_x(t, x = 1) = u_t(x = 1)$ over the whole temporal domain. We consider two distinct types of temporal boundary conditions and forcings. The set $'xsin'$ used in the main part of this study:

$$u_t^{xsin}(x) = \sin(3\pi x) \cdot x, \tag{2}$$

$$F^{xsin}(t) = \sin(4\pi t). \tag{3}$$

The results for a second set $'poly'$, based on polynomials, are presented mainly in the Supplemental Material [27]:

$$u_t^{poly}(x) = 50x(0.3 - x)(0.6 - x)(1.0 - x), \tag{4}$$

$$F^{poly}(t) = 15t(0.4 - t)(1.0 - t). \tag{5}$$

Regarding the second loss term, we consider a broad variety of applications by introducing a parametrized PDE:

$$\mathcal{D}(L, N, F) = \frac{\partial u}{\partial t} - L \cdot \frac{\partial^2 u}{\partial x^2} + N \cdot u\frac{\partial u}{\partial x} - F(t) \tag{6}$$

with parameters L and N and forcing F. The operator $\mathcal{D}$ resembles Burgers' equation for $N = 1$, for $N = 0$ $\mathcal{D}$ becomes the heat equation. The parameter L scales the linear term. The following values are taken into account:

$$L \in [0.01, 0.03, 0.1, 0.3, 1.0], \quad N \in [0.0, 1.0].$$

We trained cPINNs and qPINNs for all combinations of boundary conditions and parameterized PDEs in order to judge their potential for solving real world problems.

Networks— The cPINN architecture used is a fully connected neural network, with a given depth and width. For each investigated PDE, cPINNs with all possible combinations of depths and widths are trained and the best result is considered representatively.

The qPINN consists of a QNN and two shallow neural networks for encoding and decoding data in and from the QNN (see Fig. 5 in the Supplemental Material [27]). The parameters for the architecture are determined in a hyperparameter optimization (see Fig. 8, Supplemental Material [27]). The classical layers for en- and decoding have a width of 6 neurons. Classical network depths of 0 and 1 are tested with each PDE, and the better result is considered representatively. The quantum circuit has 3 qubits, and data is encoded before every variational layer. The circuit depth is adapted to the number of trainable parameters, which is set to 100, 150, 200 or 250 for both, cPINNs and qPINNs, to investigate various expressivities.

Methods— Two adaptations are made for the training of all PINNs in comparison to standard regression tasks. First, since training data, called collocation points, can be sampled easily, the training data set is resampled whenever

$$\mathcal{L}^{validation} > 1.1 \cdot \mathcal{L}^{train}, \tag{7}$$

where $\mathcal{L}^{train}$ is the loss for seen, and $\mathcal{L}^{validation}$ for unseen data. This prevents the PINN from overfitting and therefore enables investigating the full potential of each network (see Fig. 6 (a) in the Supplemental Material [27]). Second, there is no unambiguous choice for the training weights in Eq. (1). As it turns out, most ratios of weights eventually result in the same performance, as visualized in Fig. 6 (b) in the Supplemental Material [27]. However, a suitable choice of weights increases the convergence rate and therefore decreases the number of necessary training epochs. To avoid tuning the ratio before every training, an adaptive weighting strategy [28] is used. This leads to results similar to those obtained with optimized weighting and, more importantly, the results are independent of the choice of weights, minimizing the dependence on boundary conditions and PDE.

Incorporating the two algorithms in the training process enables reaching the full potential of the trained network as efficiently as possible.

Training & evaluation— The training process of both, cPINNs and qPINNs, with the same number of trainable parameters, is exemplarily visualized in Fig. 1. Due to the adaptive weight balancing, the loss is no suitable performance measure for the approximation anymore. For this reason, the mean squared error (MSE, calculated every 100 epochs) between exact solution obtained by numerical simulation and the PINN approximation is used representatively. Although both networks reach a similar total MSE, the qPINN needs roughly 10 times fewer training epochs. For some approximation accuracies (MSE $\sim 10^{-4}$), the qPINN is close to 100 times faster. Consequently, when training for a fixed number of epochs, the qPINN approximation is more accurate, in

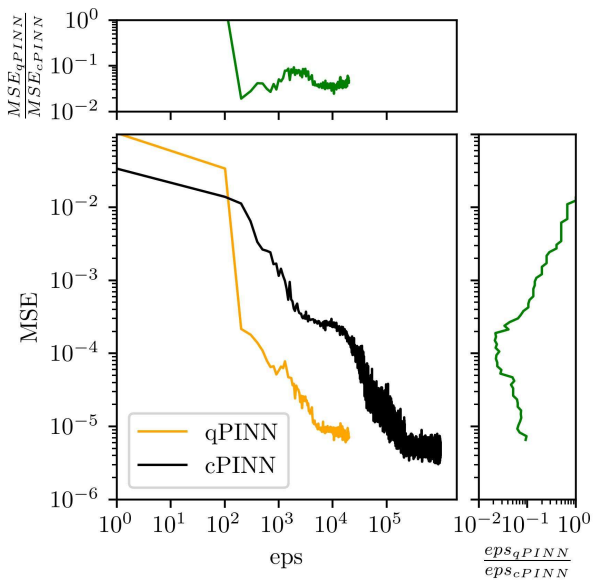


FIG. 1: Visualization of the median MSE over 10 training runs as a function of the number of training epochs for networks with 250 parameters trained on 1024 training points, for a PDE defined by $L = 0.1, N = 1.0$ with u_t^{xsin} and F^{xsin} for boundary condition and forcing. The epoch ratio to reach a certain MSE (right) and MSE ratio per epoch (top) are also shown. MSEs are calculated every 100 epochs.

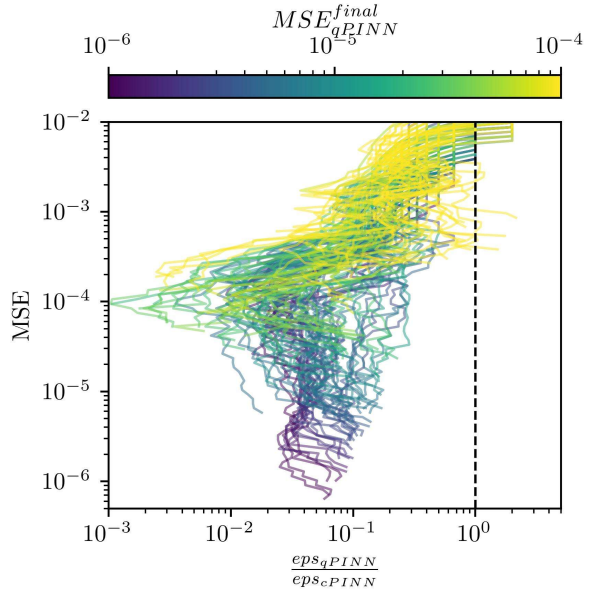


FIG. 2: Ratio of epochs needed to reach a given MSE when training cPINNs and qPINNs. The figure presents an extension of the plot on the right in Fig. 1, covering all combinations of PDEs, numbers of collocation points, and trainable parameters. Line colors indicate the MSE of the qPINN after $2 \cdot 10^4$ epochs, after which training was stopped. The dashed line marks, where cPINNs and qPINNs perform equally.

the presented instance around 50 times. However, eventually a similar MSE is reached, which is denoted as the "general accuracy limit". To investigate the cause of the improvement in training, we systematically explore all possible combinations of the boundary conditions and PDEs described above. The number of training points for both, boundary loss and PDE loss, is also varied for each combination taking values of 256, 512 or 1024. Due to the inherent variability arising from the random sampling of collocation points and trainable parameters, each PINN's performance is characterized by the median outcome across 10 independent training runs.

Results— To understand the impact of incorporating quantum circuits in networks when solving PDEs, the training results for all possible combinations of PDEs (defined by L and N), numbers of trainable parameters (100, 150, 200, 250) and number of training data (256, 512, 1024) are evaluated as exemplarily illustrated in Fig. 1 and summarized in Fig. 2 and Fig. 3.

Fig. 2 corresponds to the right plot in Fig. 1, visualizing the epoch ratio of qPINNs and cPINNs to reach certain MSEs. Some epoch ratios go down to 10^{-3} during training. Most training runs reach an epoch ratio of $\sim 10^{-2}$ before increasing again once both networks approach the general accuracy limit. As this happens earlier when the general accuracy limit is larger (bright curves) the epoch ratio is not reduced as much during training. A larger general accuracy limit is due to PDEs that are hard to solve in combination with insufficient small numbers of trainable parameters in the network. In general, the specific PDE does not appear to have a direct influence, but rather affects the general accuracy limit based on the number of parameters, which in turn is the main influences on the epoch ratio during training. This suggests that qPINNs appear to be equally beneficial for all PDEs considered as long as the number of trainable parameters is sufficient.

The training benefits from including quantum circuits in the networks, not only from the perspective of achieving a specific MSE. There is also an improvement in terms of MSE ratio for a fixed number of epochs (Fig. 3). As with the epoch ratio, the MSE ratio strongly depends on the general accuracy limit. While for larger general accuracy limits qPINNs perform only slightly better (brighter lines in Fig. 2), for small general accuracy limits the MSE of qPINNs is up to 100 times smaller than the MSE of cPINNs in the same training epoch (darker lines in Fig. 2). Nevertheless, eventually all MSE ratios seem to increase back to 1 when approaching the general accuracy limit, if they were trained sufficient long.
In summary, the general accuracy limit defines the potential of a qPINN compared to a cPINN of the same

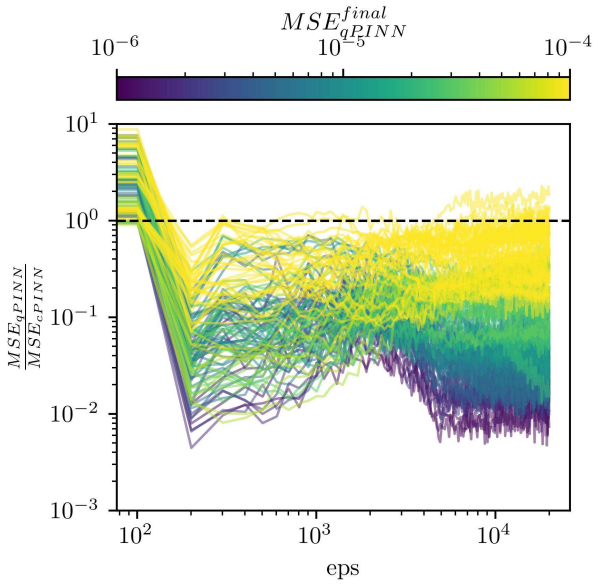


FIG. 3: Ratio of MSEs for the cPINN and qPINN as function of number of epochs trained (eps). The figure presents a summary of the plot on the top in Fig. 1, covering all combinations of PDEs, numbers of collocation points, and trainable parameters. Line colors indicate the final MSE of the qPINN for the corresponding PDE. The dashed line marks, where cPINNs and qPINNs perform equally.

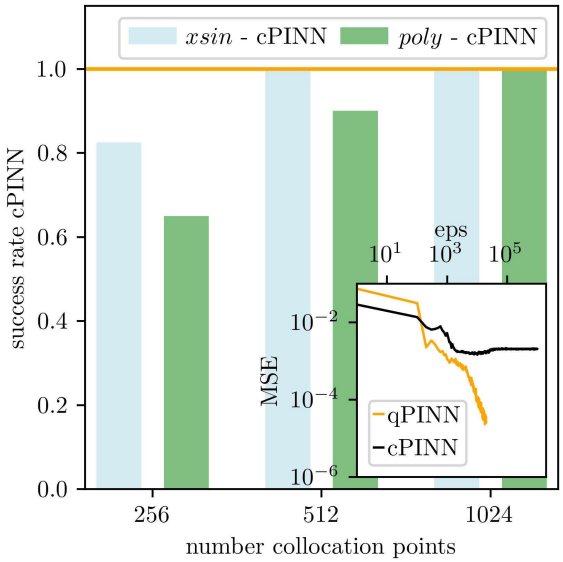


FIG. 4: Success ratio of cPINNs for various numbers of collocation points and types of boundary conditions and forcings (Eqs. 2, 3 and Eqs. 4, 5). Trainings are considered not successful when the network gets stuck in a local minimum, as visualized by the figure in the bottom right corner. When the number of collocation points is sufficiently high the training is always successful. Even though the training of qPINNs can be slowed down by reduction of training data, did not get stuck for any of the tests, as illustrated by the orange line.

size. While a larger general accuracy limit restricts the possibility of improvement, a smaller general accuracy limit enables superior performance. This is not only true for the boundary conditions 'xsin' considered here, but applies also for 'poly' which is presented in Supplemental Material [27]. Not only are qPINNs trained faster than cPINNs in terms of epochs, but their training is also more stable when the number of collocation points is small in relation to the problem complexity. In general, reducing the amount of training data can slow down the training, but the final performance does not seem to be impacted, as long as the network does not get stuck in a local minimum. As visualized in Fig. 4, for large training datasets all instances of cPINNs and qPINNs are trained successfully. However, when reducing the number of collocation points, the success ratio for cPINNs is reduced, which can be caused by the cPINNs getting stuck in a local minimum, limiting their performance. Meanwhile, qPINNs are trained even with fewer collocation points. *Discussion*—The results presented demonstrate that qPINNs train significantly faster than cPINNs. The acceleration depends on the ability of the network to find a well fitting solution eventually, characterized by the general accuracy limit. Potentially, this is caused by the number of approximate solutions that achieve a given MSE. Considering the space of approximate but imperfect solutions (for fixed boundary conditions and PDE), only a small subset corresponds to an error near the general accuracy limit. The lower the limit, the smaller this subset becomes. Consequently, finding parameter corresponding to this subset during training becomes more demanding. Hence, the apparently efficient exploration of the loss landscape by qPINNs is more beneficial the lower the general accuracy limit. The larger the general accuracy limit becomes, the larger the number of possible, imperfect solutions in the loss landscape. Since there are as many configurations of trainable parameters corresponding to these solutions, the training becomes less demanding and the efficient exploration by qPINNs less beneficial. The more efficient exploration of the loss landscape is also emphasized by the more stable training of qPINNs when little training data is available. Further investigations will be carried out in future work to confirm this explanation as well as addressing the effects of boundary conditions and PDEs. Another important property is the scaling of the presented effects in real world applications with e.g. network capacity, more complex domains or higher dimensions. Especially when considering more difficult problems in higher dimensions an

even more complex loss landscape can be expected and more training data needed, resulting in a high potential for qPINNs and their efficient exploration of loss landscapes (as indicated in Fig. 7 in the Supplemental Material [27]) and trainability on few training data. Consequently, qPINNs could become useful when dealing with real world problems, involving complex geometries, coupled PDEs and multiscale problems. For meaningful application of qPINNs in such problems, a much higher expressibility is needed, therefore requiring larger networks in terms of trainable parameters. As these networks can not be trained by state vector simulations on classical computers, a transition to real hardware is necessary to investigate the scaling behavior, especially in the possible presence of barren plateaus. For this, shot noise and its impact on the results have to be taken into account.

Conclusion—The paper shows that integrating variational quantum circuits in PINNs is beneficial for their training. By using classical layers for encoding and decoding data into quantum circuits, a suitable basis can be found to access their full capabilities. As this leads to considerably improved training efficiency, qPINNs offer great potential for the application in data-driven, complex problems, not only outperforming cPINNs but possibly also iterative numerical methods. Therefore, it is valuable to investigate whether, e.g. climate models can be enhanced by integrating observational data and parametrizations directly into the loss function while solving the underlying PDEs by qPINNs. This approach is promising to address the computationally costly parameter tuning process, thereby replacing separate, explicit optimization procedures.

Acknowledgments—This project was made possible by the DLR Quantum Computing Initiative and the Federal Ministry of Research, Technology and Space; `qci.dlr.de/projects/klim-qml`. V.E. was additionally supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) through the Gottfried Wilhelm Leibniz Prize awarded to Veronika Eyring (Reference No. EY 22/2-1). This work used resources of the Deutsches Klimarechenzentrum (DKRZ) granted by its Scientific Steering Committee (WLA) under project ID bd1179.

[1] E. Suarez, H. Bockelmann, N. Eicker, J. Eitzinger, S. El Sayed, T. Fieseler, M. Frank, P. Frech, P. Giesselmann, D. Hackenberg, G. Hager, A. Herten, T. Ilsche, B. Koller, E. Laure, C. Manzano, S. Oeste, M. Ott, K. Reuter, and B. Vieth, Frontiers in High Performance Computing **3** (2025), 10.3389/fhpcp.2025.1520207.

[2] P. Fischer, M. Min, T. Rathnayake, *et al.*, The International Journal of High Performance Computing Applications **34**, 562 (2020).

[3] M. W. M. G. Issanayake and N. Phan-Thien, Communications in Numerical Methods in Engineering **10**, 195 (1994).

[4] Y. Wang, C.-Y. Lai, C. Cowen-Breen, and et al., "Discovering the rheology of antarctic ice shelves via physics-informed deep learning," `https://doi.org/10.21203/rs.3.rs-2135795/v1` (2022), pREPRINT (Version 1) available at Research Square.

[5] M. Baldan, P. Di Barba, and D. A. Lowther, IEEE Transactions on Magnetics **59**, 1 (2023).

[6] M. A. Giorgetta, R. Brokopf, T. Crueger, M. Esch, S. Fiedler, J. Helmert, C. Hohenegger, L. Kornblueh, S. Rast, D. Reinert, M. Sakradzija, M. Schubert-Frisius, H. Wan, G. Zaengl, and B. Stevens, Journal of Advances in Modeling Earth Systems **10**, 1635 (2018).

[7] P. Bonnet, L. Pastori, M. Schwabe, M. Giorgetta, F. Iglesias-Suarez, and V. Eyring, Geoscientific Model Development **18**, 3681 (2025).

[8] D. Kim and J. Lee, Multiscale Science and Engineering **6**, 1 (2024).

[9] M. Schwabe, L. Pastori, I. de Vega, P. Gentine, L. Iapichino, V. Lahtinen, M. Leib, J. M. Lorenz, and V. Eyring, Environmental Data Science **4** (2025), 10.1017/eds.2025.10010.

[10] Y. Wang, C.-Y. Lai, J. Gómez-Serrano, and T. Buckmaster, Phys. Rev. Lett. **130**, 244002 (2023).

[11] Y. Wang, M. Bennani, J. Martens, S. Racanière, S. Blackwell, A. Matthews, S. Nikolov, G. Cao-Labora, D. S. Park, M. Arjovsky, D. Worrall, C. Qin, F. Alet, B. Kozlovskii, N. Tomašev, A. Davies, P. Kohli, T. Buckmaster, B. Georgiev, J. Gómez-Serrano, R. Jiang, and C.-Y. Lai, "Discovery of unstable singularities," (2025), arXiv:2509.14185 [math.AP].

[12] T. G. Grossmann, U. J. Komorowska, J. Latz, and C.-B. Schönlieb, IMA Journal of Applied Mathematics **89**, 143 (2024), https://academic.oup.com/imamat/article-pdf/89/1/143/58325885/hxae011.pdf.

[13] P.-Y. Chuang and L. A. Barba, "Experience report of physics-informed neural networks in fluid simulations: pitfalls and frustration," (2022), arXiv:2205.14249 [physics.flu-dyn].

[14] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. Latorre, Quantum **4**, 226 (2020).

[15] O. Kyriienko, A. E. Paine, and V. E. Elfving, Phys. Rev. A **103**, 052416 (2021).

[16] S. Berger, N. Hosters, and M. Möller, Scientific Reports **15**, 18823 (2025), published 2025-05-29.

[17] P. Siegl, S. Wassing, D. M. Mieth, S. Langer, and P. Bekemeyer, CEAS Aeronautical Journal **16**, 63 (2025).

[18] C. Trahan, M. Loveland, and S. Dent, Entropy **26** (2024), 10.3390/e26080649.

[19] A. Sedykh, M. Podapaka, A. Sagingalieva, K. Pinto, M. Pflitsch, and A. Melnikov, Machine Learning: Science and Technology **5**, 025045 (2024).

[20] A. Abbas, D. Sutter, C. Zoufal, A. Lucchi, A. Figalli, and S. Woerner, Nature Computational Science **1**, 403–409 (2021).

[21] A. Abbas, D. Sutter, A. Figalli, and S. Woerner, "Effective dimension of machine learning models," (2021), arXiv:2112.04807 [cs.LG].

[22] M. Schuld, R. Sweke, and J. J. Meyer, Physical Review A **103** (2021), 10.1103/physreva.103.032430.

[23] M. C. Caro, H.-Y. Huang, M. Cerezo, K. Sharma, A. Sornborger, L. Cincio, and P. J. Coles, Nature Communications **13**, 4919 (2022).

[24] J.-P. Liu, H. O. Kolden, H. K. Krovi, N. F. Loureiro, K. Trivisa, and A. M. Childs, Proceedings of the National Academy of Sciences **118** (2021), 10.1073/pnas.2026805118.

[25] S. Succi, W. Itani, C. Sanavio, K. R. Sreenivasan, and R. Steijl, Computers & Fluids **270**, 106148 (2024).

[26] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient pathologies in physics-informed neural networks," (2020), arXiv:2001.04536 [cs.LG].

[27] "See supplemental material at [url will be inserted by publisher] for information about the specifications of the pinn implementation, results for further boundary conditions, hyperparameter studies and training insights for qpinns." `URL_will_be_inserted_by_publisher` (2025).

[28] S. Wang, S. Sankaran, H. Wang, and P. Perdikaris, "An expert's guide to training physics-informed neural networks," (2023).

[29] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, Phys. Rev. A **99**, 032331 (2019).

[30] I. Sobol', USSR Computational Mathematics and Mathematical Physics **7**, 86 (1967).

[31] B. Jaderberg, A. A. Gentile, Y. A. Berrada, E. Shishenina, and V. E. Elfving, Phys. Rev. A **109**, 042421 (2024).

[32] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, C. Blank, K. McKiernan, and N. Killoran, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," (2018), arXiv:1811.04968 [quant-ph], 1811.04968.

[33] P. Kidger and C. Garcia, Differentiable Programming workshop at Neural Information Processing Systems 2021 (2021).

[34] DeepMind, I. Babuschkin, K. Baumli, A. Bell, S. Bhupatiraju, J. Bruce, P. Buchlovsky, D. Budden, T. Cai, A. Clark, I. Danihelka, A. Dedieu, C. Fantacci, J. Godwin, C. Jones, R. Hemsley, T. Hennigan, M. Hessel, S. Hou, S. Kapturowski, T. Keck, I. Kemaev, M. King, M. Kunesch, L. Martens, H. Merzic, V. Mikulik, T. Norman, G. Papamakarios, J. Quan, R. Ring, F. Ruiz, A. Sanchez, L. Sartran, R. Schneider, E. Sezener, S. Spencer, S. Srinivasan, M. Stanojević, W. Stokowiec, L. Wang, G. Zhou, and F. Viola, "The DeepMind JAX Ecosystem," (2020).

[35] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," (2018).

Supplemental Material

Physics-informed neural networks— In general, a PDE can be defined by

$$\mathcal{D}(u) = 0 \tag{8}$$

where $\mathcal{D}$ is a differential operator and $u(t, x)$ is a function depending on space x and time t. Because the PDE itself only describes the change of quantities u in space and time, some absolute information, which are typically in the form of temporal boundary conditions u_t and spatial boundary conditions u_x, have to be given as well

$$u(t,x) \quad = u_t(x), \qquad (t,x) \in [0] \times \Omega, \tag{9}$$

$$u(t,x) \quad = u_x(t,x), \qquad (t,x) \in [0,T] \times \partial\Omega, \tag{10}$$

where Ω and $\partial\Omega$ denote the spatial domain and its boundaries in x (which can be multidimensional) and T the temporal domain. However, for PINNs providing arbitrary fixed points works as well. To solve a PDE by using a neural network, the network takes temporal and spatial coordinates t, x as inputs and outputs an approximation of the solution. Here, the network $f(t, x; \boldsymbol{\theta})$ can be any differentiable and parametrized network/function. If $\boldsymbol{\theta}$ were adjusted so that $f(t, x; \boldsymbol{\theta}) = u$ is the optimal solution, Eqs. (8), (9) and (10) would be given exactly. Therefore, a loss function $\mathcal{L}$ to be minimized can be defined as

$$\begin{aligned}
\mathcal{L}_{\boldsymbol{\theta}} &= \mathcal{L}_{pde} + \mathcal{L}_t + \mathcal{L}_x \qquad (11)\\
\mathcal{L}_{pde} &= \sum_{(t,x)\in[0,T]\times\Omega} |\mathcal{D}(f(t,x|\boldsymbol{\theta}))|^2\\
\mathcal{L}_t &= \sum_{(t,x)\in[0]\times\Omega} |f(t,x|\boldsymbol{\theta}) - u_t(x)|^2\\
\mathcal{L}_x &= \sum_{(t,x)\in[0,T]\times\partial\Omega} |f(t,x|\boldsymbol{\theta}) - u_x(t,x)|^2.
\end{aligned}$$

While the terms $\mathcal{L}_t$ and $\mathcal{L}_x$ are standard regression tasks, $\mathcal{L}_{pde}$ relies on the differentiability of $f(t, x|\boldsymbol{\theta})$, i.e. $\frac{\partial f(t,x|\boldsymbol{\theta})}{\partial t}$ which approximates $\frac{\partial u}{\partial t}$ for a parameter set $\boldsymbol{\theta}$, containing all the trainable parameters. This enables estimating the solution in the domains where no actual data is given. Since both classical neural networks and quantum circuits [29] are differentiable in that sense, they can be used to approximate the solution u to a given PDE. In this study, the temporal and spatial boundary conditions are treated equivalently (as they are mathematically similar) by defining $\mathcal{L}_{bounds} = \mathcal{L}_t + \mathcal{L}_x$. Also using different weightings, for the two parts of Eq. (11), effectively changing the loss landscape, turns out to be beneficial for the training process (Fig. 6 (b)). However, in general the weights do not only depend on the network size and problem, but they also can be tuned only based on trained networks and a known exact solution.

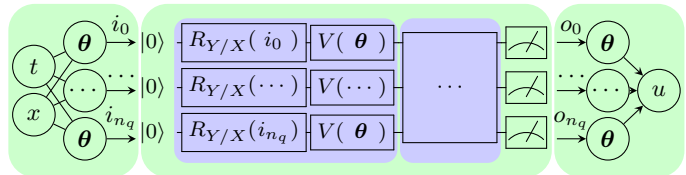


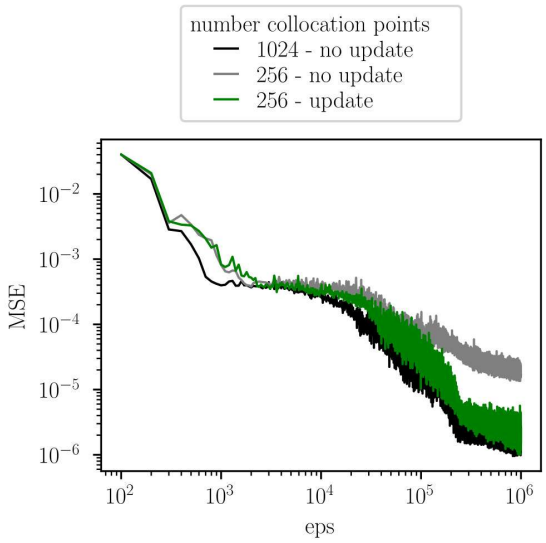
FIG. 5: Schematic representation of a hybrid neural network architecture with n_q qubits used as qPINN. The spatial coordinate x and time t are input to a classical dense neural network (left block) depending on a set of parameters $\boldsymbol{\theta}$. The outputs of the classical network i_j are encoded into a quantum circuit (center block) through alternating rotational gates $R_{Y/X}(i_j)$ around the Y and X axes. Then, parametrized single-qubit operations followed by entangling operations, both represented by $V(\boldsymbol{\theta})$, are applied. These encoding and parametrized layer sequences (marked in purple) can be repeated multiple times to increase expressivity. Expectation values of the Pauli-Z operator are measured for each qubit, and the resulting quantum outputs o_j are fed into a second classical network to produce the final output u (right block).

As this not practicable in a real application, an algorithm balancing the different terms is used as described above [28], resulting in a training process close to optimally tuned weights. Based on this, $f(t, x|\boldsymbol{\theta})$ approximates u by finding a set of parameters

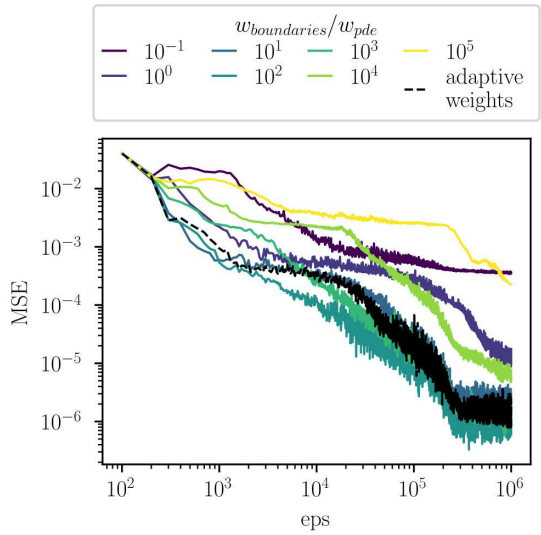
$$\boldsymbol{\theta} = \arg\min_{\boldsymbol{\theta}^*} \mathcal{L}_{\boldsymbol{\theta}^*}. \tag{12}$$

Training settings— If not otherwise stated, $2 \cdot 10^4$ epochs are used for training the qPINNs and $1 \cdot 10^6$ for cPINNs. The learning rate for the Adam optimizer is initially set to 0.01 and gradually reduced by a factor up to 0.1 during training. To evaluate Eq. (11), so-called collocation points are randomly sampled from $[0, T] \times \Omega$, $[0] \times \Omega$ and $[0, T] \times \partial\Omega$ to calculate the individual loss terms. 256, 512 or 1024 points are sampled for both, boundaries and domain by Sobol sampling [30]. In each case 4 batches are used during training. Based on the calculated loss, the trainable parameters in the networks are updated by an Adam optimizer. A training example is visualized in Fig. 1.

Updating training data— To enable networks to learn the optimal solution and reach their full potential, sufficient training data is required. While for regression tasks the training data is often limited, this can be circumvented in PINNs by generating additional collocation points. One strategy is to use a fixed larger data set throughout the training process. Alternatively, especially for high-dimensional or complex systems, smaller sets can be used and updated once the network has effectively learned the current data and begins to over-



(a) Mean squared error (MSE) for cPINNs calculated with 1024 and 256 collocation points, with and without updating the training data



(b) Mean squared error (MSE) for cPINNs calculated for various numbers of weight ratios $w_{boundaries}/w_{pde}$, resp. with an adaptive weighting algorithm as function of training epochs eps.

FIG. 6: Effect of updating training data (a) and choice of weights (b). All trainings were conducted with 250 trainable parameters, a PDE defined by $L = 0.1$, $N = 0.0$ with u_t^{xsin} and F^{xsin} for boundary condition and forcing.

fit, as described in Eq. 7. The options are compared in Fig. 6 (a). Using 256 collocation points without update limits the solution accuracy compared to 1024 collocation points. When using 256 collocation points, the accuracy is the same as when using 1024, enabling the utilizing the

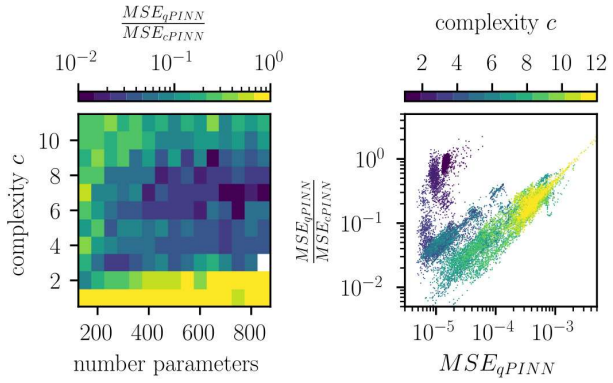


FIG. 7: MSE ratio after 5000 epochs for various complexities and trainable parameters are visualized on the left. On the right, the MSE ratio is shown depending on the respective MSE of the qPINN for various complexities.

networks full potential.

Adaptive weights— The effect of using adaptive weights instead of fixed weights in Eq. 1 and Eq. 11, as mentioned in the main part, is visualized in Fig. 6 (b). The adaptive weights are defined as the inverse of the norm of the respective loss gradient with respect to the trainable parameters [28]. As illustrated by the dashed lines, this sufficiently resembles an optimal choice of weights by adaptively balancing the loss terms. While using adaptive weights may not always be the optimal configuration, it eliminates the need to tune fixed weights, a process that is impractical in real-world applications, since the true solution would have to be known. More important, it also prevents the results from being dependent on the PDE under consideration, as they might benefit from different fixed weights.

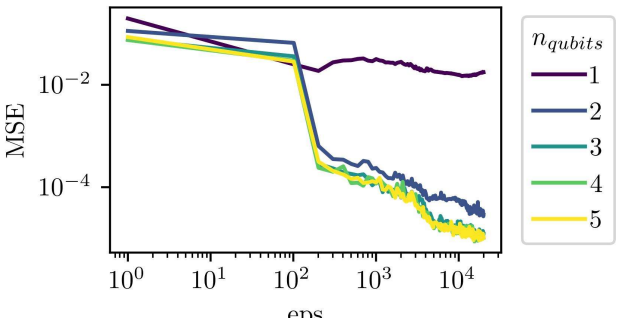
Scaling with complexity—To get an impression, how the benefits of qPINNs might scale with problem complexity, trainings with parameterized boundary conditions u_t^c and forcings F^c were conducted:

$$u_t^c(x) = \sin(c \cdot \pi x) \cdot x, \quad F^c(t) = \sin(c \cdot \pi t). \tag{13}$$

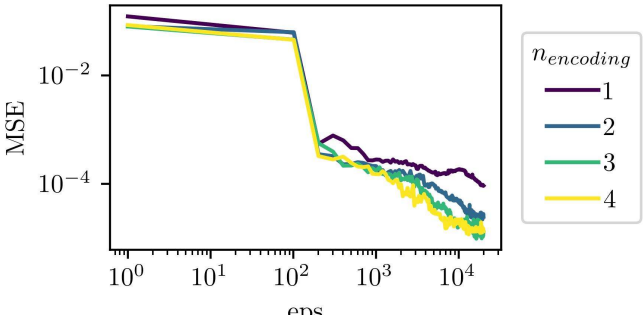
The advantage of qPINNs compared to cPINNs in terms of the MSE ratio is visualized in Fig. 7. The left figure illustrates, that the efficiency of qPINNs (here measured by the MSE ratio) scales with the problem complexity when the number of trainable parameters is sufficient. This is visible by the minimal MSE ratio becoming smaller with increasing complexity. The right figure indicates not only, that the dependency between MSE of the qPINN and MSE ratio in a given epoch follows roughly a power law, but also, that this relation shifts to smaller MSE ratios with increasing complexity. Although no generalization is possible, the results indicate, that the advantage of qPINNs due to accelerated training might scale with the problem complexity.

Architectures— The purely classical network (cPINN) used in this study is a dense neural network. As activation function tanh is applied. The number of nodes is the same for all hidden layers. For a given number of trainable parameters $\boldsymbol{\theta}$ all possible numbers of hidden layers are tested to find the optimal configuration. The number of nodes per hidden layer is adjusted so that number of trainable parameters is matched as closely as possible. After calculating the MSE for all possible network depths, the depth resulting in the lowest MSE is chosen in the manuscript to represent the network size.
The architecture used for incorporating quantum circuits into a hybrid network (qPINN) is inspired by two methods. First, the hardware efficient design of the quantum circuit itself is chosen as in [22]. Second, to make efficient use of the Fourier frequencies, the input to the quantum circuit is transformed into a meaningful basis by a classical network [31]. Consequently, it is not necessary to consider different encoding types [15]. Finally, a second classical network maps the output of the quantum circuit to the solutions (Fig. 5).
The parameters specifying the hybrid network were obtained from a hyperparameter study visualized in Fig. 8. We found that the number of qubits does not matter as long as the number of trainable parameters remains the same (and as long as features can be encoded sufficiently, Fig. 8(a)), which is in agreement with the spectrum size described in [22]. Therefore, 3 qubits are used to reduce the computational effort to simulate the quantum circuit. The final MSE benefits from encoding data as often as possible for a given number of parameters, as shown in Fig. 8 (b). Here, each encoding means encoding in R_Y and R_X, as illustrated in Fig. 5. When optimizing the classical input and output layers (Fig. 8 (c)) it becomes obvious, that the approximation benefits from using as much quantum resources as possible, consequently minimizing the parameters in classical layers. To guarantee a suitable encoding basis for each PDE considered in this work, two qPINNs are tested for each PDE and the better result of the two is used representatively, analogous to the cPINN. The first qPINN has no hidden layers, the second qPINN uses one hidden layer with 6 neurons in the input and output network.

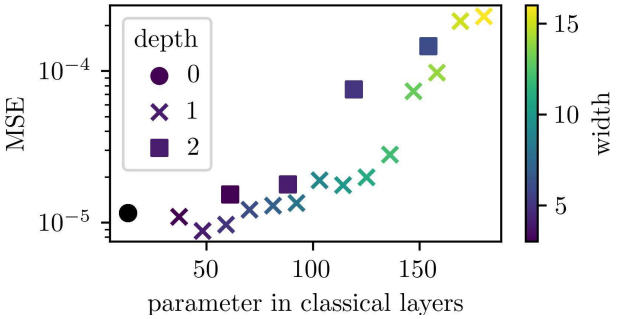
Implementation— For implementing these networks, the python libraries pennylane [32], equinox [33], optax [34] and jax [35] were used. Equinox enables easy integration of quantum circuits defined in pennylane in a differentiable network. Therefore the same training workflow, based on optax, can be applied to train all networks. jax accelerates the training process. In this study, shot noise is not taken into account, as the primary focus is on exploring the fundamental capabilities enabled by integrating differentiable quantum circuits into PINNs.



(a) Mean squared error (MSE) after $2 \cdot 10^4$ epochs as a function of the number of trained epochs for various numbers of qubits in the quantum circuit.



(b) Mean squared error (MSE) as a function of the number of trained epochs for various numbers of encodings from the classical input layer into the quantum circuit.



(c) Mean squared error (MSE) as a function of the number of trained epochs for various width and depth of classical input and output layers.

FIG. 8: Default values used in the tests are $n_{qubits} = 3$, $depth = 1$, $width = 6$ and data is encoded into the circuit as often possible. The total number of trainable parameters in the circuit is 200 and the amount of training data is 512 for each, boundary and PDE loss. The considered PDE uses $L = 0.1, N = 1.0$ with 'xsin' boundary conditions and forcing.

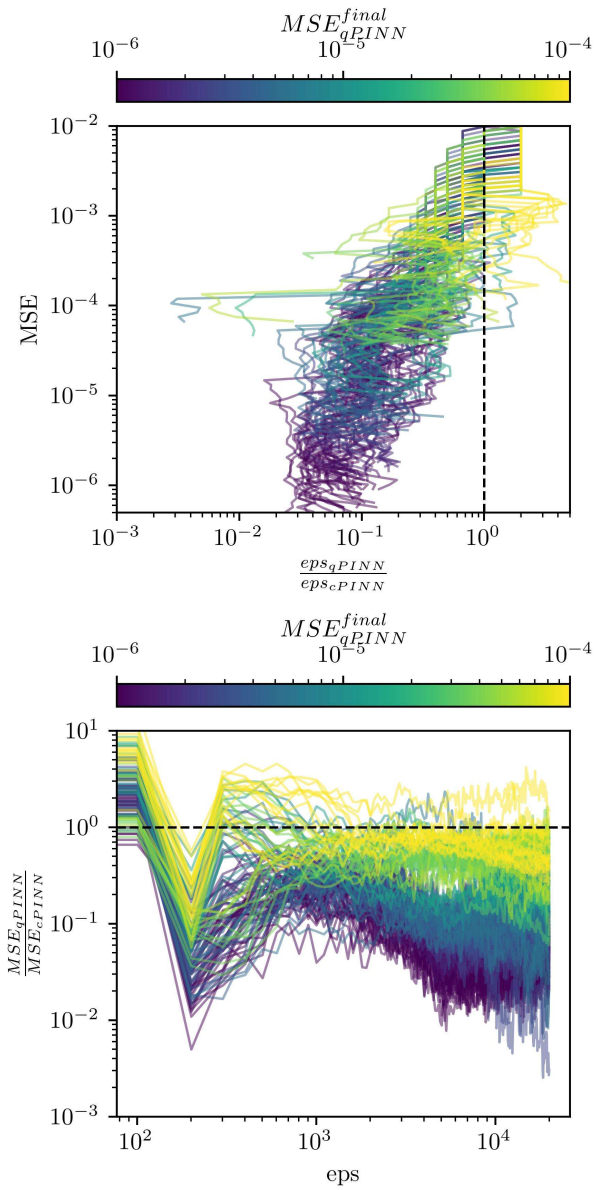


FIG. 9: MSE as a function of epoch ratios (top) and MSE ratios as a function of epochs trained (bottom) when training 'poly' boundary conditions and forcings. The line colors indicate the final MSE of the qPINN for the corresponding PDE. The dashed lines mark, where cPINNs and qPINNs perform equally.

Results for further boundary conditions and forcings— Similar to Fig. 2 and Fig. 3 for 'xsin', Fig. 9 visualizes the training progress of cPINNs and qPINNs for 'poly' boundary conditions and forcings. Again, the general accuracy limit has a strong influence on epoch ratios and MSE ratios, which show similar behavior as for 'xsin'. This indicates that qPINNs can offer benefits in various use cases.

Trained basis—To illustrate the contribution of each

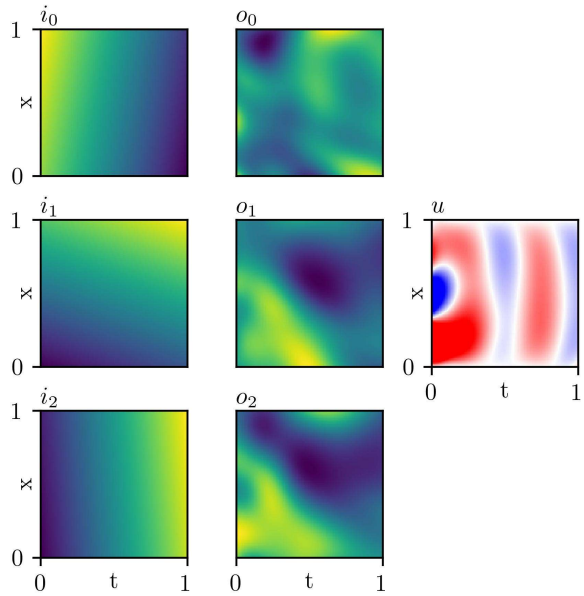


FIG. 10: Individual outputs of the hybrid architecture (as visualized in Fig. 5) for a hybrid network based on a three-qubit quantum circuit. The total number of trainable parameters in the circuit is 200 and the amount of training data is 512 for each, boundary and PDE loss. The considered PDE uses $L = 0.1, N = 1.0$ with 'xsin' boundary conditions and forcing. i_0 and i_1 represent the output of the classical input network. Since this is a qualitative visualization, no actual values corresponding to the colors are provided.

of the three parts from Fig. 5, classical input network, quantum circuit, classical output network, their individual outputs are presented in Fig. 10. In this example, i_0 and i_2 tends to encode the t-coordinate, while i_1 encodes the sum of the t-coordinate and x-coordinate. This relates to the solution, which changes more across time than space, especially for later times. Obviously this can change significantly, when domain becomes much more complex than the square considered here, consequentiality requiring more qubits and larger classical networks to find and encode a suitable basis. In the present case, the quantum circuit outputs o_0, o_1, and o_2 are based on the learned basis, which seems to be an efficient decomposition of the solution u to be combined by the second classical network.

Path in loss landscape— To get an impression of the reason for the different convergence rates of cPINNs and qPINNs the evolution of randomly chosen parameter pairs are traced during training and visualized in Fig. 11. It is clearly visible that the path of the cPINN's parameter pair is more chaotic while the qPINN's parameter pair moves much more targeted in the direction of final parameter values, making it more efficient. This seems to be made possible by a landscape that has a clearly

recognizable and fixed minimum. This could potentially be caused by the entanglement in the quantum circuit, which connects the trainable parameters more strongly to each other than trainable parameters are connected in a classical network. Consequently, the impact on the landscape of other parameters when updating a given parameter can be minimized, resulting in a more stable and well-behaved optimization landscape for each of the parameters and therefore faster training procedure. This is visible in Fig. 11, where the loss landscape of the cPINN changes during the training, while the qPINN's loss landscape tends to remain constant. Even though further research is needed to confirm this explanation, being able to find an efficient path towards the optimal parameter configuration is crucial when the problems - and consequently the loss landscapes - get far more complex, this further indicates, that qPINNs are a promising method to approach especially these applications more efficiently.

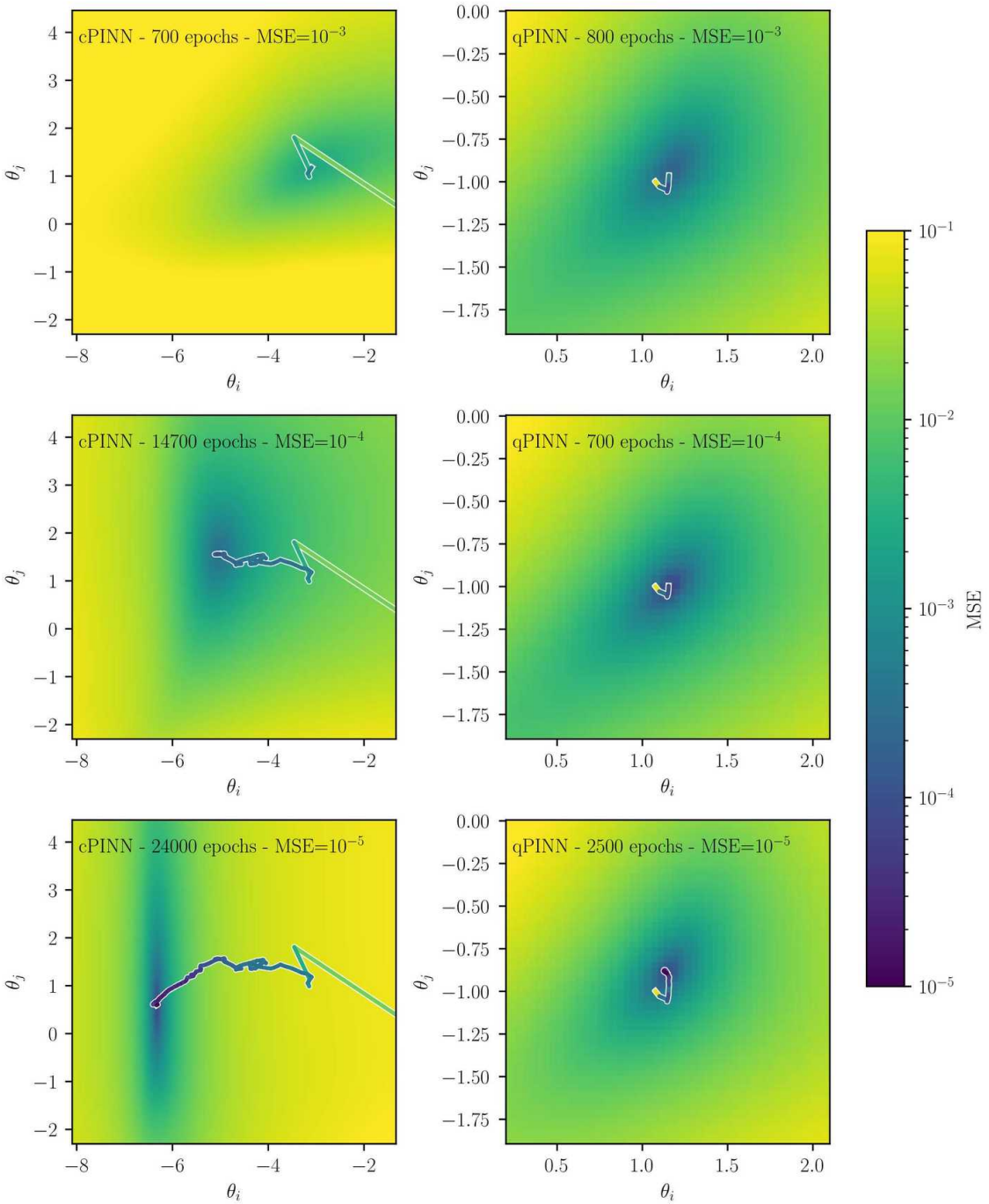


FIG. 11: The evaluation of two trainable parameters during training is illustrated when reaching a MSE of 10^{-3}, 10^{-4}, and 10^{-5} for a cPINN (left) respectively a qPINN (right). The lines track the parameter values from the initial value up to the visualized epoch. While for the cPINN the two parameters are weights from neighboring nodes from the first hidden layer, for the qPINN the parameters are two neighboring parameters from the first variational layer of the quantum circuit. The background image is obtained by calculating the MSE when varying the parameters $\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$ in the respective epoch. The total number of trainable parameters in the circuit is 250 and the amount of training data is 1024 for each, boundary and PDE loss. The considered PDE uses $L = 0.1, N = 1.0$ with 'xsin' boundary conditions and forcing.